School of Computer and Information Sciences
COMP2240/COMP6240 - Operating Systems

Assignment 3 (20%)

Submit using Canvas by **11:59 pm, Sunday 8th November 2026**

1. Introduction

In this assignment you are once again to produce a simulation – this time you will be simulating Paging with Virtual Memory, with the Clock Page Replacement Policy under the Fixed Allocation with Local Replacement Resident Set Management Policy. You will be required to produce a small Java application demonstrating the simulation of this policy, as well as a written report documenting aspects of your code and results.

You are advised to make yourself familiar with the Submission Requirements (*Section 3*), and assignment expectations before beginning the coding task.

2. Assignment Task

In Assignment 1, we assumed that the system had *an infinite amount of memory*. More realistically an operating system has access to a limited amount of memory, and this needs to be managed to meet process demands, possibly with page replacements.

You will write a program to simulate a system that uses paging with virtual memory. You are to evaluate the performance of the **Clock** page replacement algorithm for the **fixed allocation with local replacement** strategy, as introduced in lectures.

The system characteristics are described below:

2.1. Memory

The system has `F` frames available in user memory space, a value that will be supplied as an input argument. During execution, the processor will determine if the page required for the currently running process is in main memory.

a. If the page is in main memory, the processor will access the instruction and continue.
b. If the page is not in main memory, the processor will issue a *page fault* and *block* the process until the page has been transferred to main memory.
c. Initially no page is in the memory, i.e., the simulation will be strictly using ***demand paging***, where pages are only brought into main memory when they are requested.
d. In fixed allocation scheme frames are equally divided among processes, additional frames remain unused.

2.2. Paging & Virtual Memory

The system using paging with following characteristics.

a. Each process can have **a minimum 1 and maximum 50 page references** where each page contains a single instruction. Page numbers are local to each process, i.e., page-1 of Process-1 is different from page-1 of Process-2.
b. For Page Replacement you will need to apply **Clock** policy as taught in this course. The Clock policy works using a circular buffer as follows:
    i. Every frame has a use bit. The use bit of a frame is set to 1 when a page is loaded into the frame, and each time the page in the frame is referenced by its process (i.e., on a page hit).
    ii. A pointer (the clock hand) moves around the frames in circular order, starting from the first frame. To choose the frame for an incoming page, the frame under the pointer is examined: if the frame is empty or its use bit is 0, this frame is chosen; otherwise its use bit is set to 0 and the pointer moves to the next frame, and so on. After a frame has been chosen, the pointer moves to the next frame. The pointer does not move on a page hit.
    iii. In the fixed allocation scheme each process has its own pointer over its own frames (every pointer starts at the first frame of the process).
c. You will need to implement the ***Fixed Allocation with Local Replacement Scope*** scheme for resident set management. In this scheme, the frames allocated to a process do not change over the simulation period, i.e., even after a process finishes (and the allocated frames to that process become free), the allocated frames to other processes do not change. And the page replacements (if necessary) will occur within the frames allocated to a process using **Clock** policy.
d. Each page reference of a process is either a read (R) or a write (W). A page that has been written becomes dirty and has to be written back to disk before its frame can be reused (see Dirty Pages & Write-back below).

2.3. Page Fault Handling

When a page fault occurs, the interrupt routine will handle the fault by blocking that process and placing an I/O request. The I/O controller will process the I/O request and bring the page into main memory. This may require replacing a page in main memory using the **Clock** page replacement policy.

Other ready processes should be scheduled to run while such an I/O operation is occurring.

a. Issuing a page fault and blocking a process takes *no time*, so *multiple page faults may occur* and then another ready process can run immediately *at the same time unit*.
b. Swapping in a page takes *4 units* of time (*if a page required by a process is not in main memory, the process must be put into its blocked state until the required page is available*).
c. If a process is unblocked (*i.e. the requested page is placed in the main memory*) at time *t* then it can be scheduled and the requested page can be executed at *t*.

d. The frame for the incoming page, and so the victim page if a page has to be replaced, is chosen when the swapping in of the page is complete, i.e. 4 time units after the page fault was issued, not when the page fault is issued.

2.4. Dirty Pages & Write-back

Every page reference of a process is either a read (R) or a write (W). A page is clean when it is loaded into a frame, and it becomes dirty when a write to it is executed. A dirty page has to be written back to disk before its frame can be used for another page.

a. Replacing a clean page takes no additional time. Replacing a dirty page takes 1 additional time unit for the write-back, so the process that caused the page fault is blocked for 4+1 = 5 time units instead of 4 if that incoming page is replacing a dirty page.
b. When a process has executed its last page and it still has dirty pages in memory, they are written back in parallel. This takes 1 time unit: the process releases the CPU at once, but it is finished one time unit later. That is, if its last page is executed at time t, its turnaround time is t+2 (t+1 if it has no dirty page).
c. The number of write-backs of a process ("# Write-Backs" in the output) is the number of times it waited for a write-back: one for each dirty page replaced for its page faults, plus one if it finished with dirty pages.

2.5. Scheduling

The system is to use a **Round Robin short-term scheduling algorithm** with time a quantum of ***Q,*** a value that will be supplied as an input argument.

a. Executing a single instruction (*i.e. a page*) takes *1 unit* of time.
b. Switching the processes *does not take any time*.
c. All the processes start execution at time *t = 0*. And they will be processed in the order the *process names appear in the input*.
d. If a process becomes ready at time unit *t* then execution of that process may occur in the same time unit *t* without any delay (if there is no other process running or waiting in the ready queue).
e. If *multiple process* becomes ready at the same time, then they will enter the ready queue *in the order they became blocked*.
f. If a process **`P1`** is finishes it's time quantum at *t1* and another process **`P2`** becomes unblocked at the same time *t1*, then the unblocked process, **`P2`**, is added in the ready queue *first* and the *time-quantum expired* process, **`P1`**, is added *after* that.

You are to evaluate the performance of the ***Clock with Fixed Allocation & Local Page Replacement algorithm***.

Please use the basic versions of the policies introduced in the lectures.

2.6. Process File Format

Since we assume that each page contains a single instruction, an execution trace is simply a sequence of page requests, each being a read (R) or a write (W).

For example: (`Process1.txt`)

```
begin
W 1
R 2
W 3
R 4
R 5
end
```

This specifies a process that first writes page 1 (W 1), then reads page 2 (R 2), and so on until the `end` is reached.

Note 1) the file consists of begin, the references and end, where every reference is the letter R (read) or W (write), not case sensitive, followed by a page number; white space (spaces, tabs and line breaks) only separates these words and plays no other part in the file format – the above file is just as valid if supplied as a single line: begin W 1 R 2 W 3 R 4 R 5 end

Note 2) the filename has no relevance – it will be a text file, but should not be restricted to any specific extension. The processes are identified by their position in the command line: the first file is Process1, the second file is Process2, and so on (these names are used in the output).

2.7. Simulation Output

The simulation should produce a summary showing, for each process, the total turnaround time (*including I/O and write-back time*), the total number of page faults, the number of write-backs, and the time the page faults were generated.

The output should be printed to the console (*i.e.: the terminal window, from where your program was run*), using the following sample layout:

```
Clock - Fixed-Local Replacement:
Process Name      Turnaround Time  # Faults  # Write-Backs  Fault Times
Process1          30               5         1              {0, 5, 11, 18, 24}
Process2          34               5         1              {0, 6, 15, 20, 25}
Process3          17               1         0              {0}
Process4          33               4         1              {0, 10, 16, 24}
```

See `\Sample1` for the dataset that generate this output, and `\Sample2` for a second dataset and sample results. Check the README.txt file for details about the simulation condition.

It is suggested you generate some further testing data than that provided.

2.8. Report (s)

Finally, you will submit a **brief 1 page** (*A4*) **report** (named as **Report.pdf**)..

Your report should document the following:

a. Your testing approach and methodology.
b. An analysis of the results from your program (including the effect of dirty pages and write-backs on the turnaround times) and any interesting observations.
c. A discussion on edge cases you considered, and behaviour of your algorithm on those cases.
d. A review of the results from your program, including any interesting observations or unexpected behaviour you encountered during development, including any specific issues you faced and how you addressed those.

If you have used GenAI tool(s) in preparing this assignment, then you also need to write a 2 page (A4) reflective report (name as **GenAIReport.pdf**). In your reflective note, you should provide a detailed account of your experience and should include, but not limited to, the following elements:

a. overview of the tool(s) used,
b. how the tool(s) was/were integrated in your workflow,
c. what was the quality/correctness of the response generated by the tool(s) and/or how the GenAI tool(s) improved the quality of your solution,
d. how the tool(s) improved your learning experience and problem-solving skills,
e. what challenges you faced in using those tools and how you solved those challenges,
f. how useful was the tool for your productivity e.g. how much time was saved by using GenAI tool(s),
g. any ethical consideration or concerns in using GenAI tools in this assessment etc.

GenAI must not be used in preparing any report.

3. Programming Standards

Your submission must also conform to the follow requirements.

3.1. Programming Language

The programming language is Java, versioned as per the University Lab Environment (*Note this is currently **Java 25***). You may only use standard Java libraries as part of your submission.

3.2. Code Standards

Yes, *Code standards matter*! There is a mark component that covers commenting and general code form, so make sure your submitted code is consistent, valid variable names, appropriate commenting, etc.

Additionally, **there will be no nested classes*** – each java class will be in a separate file. Submitting something like ALL classes inside a single class will attract a heavier penalty, so please make sure you code your submission in a manner that keeps a single class in a single file.

*Nested Class are permitted in specific use cases, in line with Java Best Practices, such as Iterators being written as a Private Inner Class of its container. But there are no valid use cases immediately apparent for this assignment.

Yes, multiple source files can be compiled in line with the specification, as compilation of the program entry point class will invoke compilation of any referenced classes, and so forth!

3.3. User Interface

The output should be printed to the console (*ie: the terminal window, from where your program was run*); there is no need to print to file, create a menu system, or display in a graphical manner.

3.4. Input and Output

Your program will accept data from a set of input files, whose names are specified as part of a command line argument. **Hint:** the Java `Scanner` Library is something you will likely want to use here!

Your submission will be tested with the above data and will also be tested with other input files, that may or may not be formatted the same, with respect to white space!

The simulation should produce a summary showing, for each process, the turnaround time, the total number of page faults, the number of write-backs and the time the page faults were generated.

Sample inputs/outputs are provided. Working of the first example is presented with details of different levels for visualisation. Your submission will be tested with this data and will also be tested with other input files.

Your program should output to standard output (*this means output to the Console*). Output should strictly follow the provided format samples (*see the output files*). **If output is not generated in the required format, then there will be a heavy penalty.**

3.4.1. Command Line Parameters

Input to your program should be via **command line arguments** (*see also in* **Section 3.6. Deliverable:**), where the arguments are system configurations and the names of files that specify the execution trace of different processes. **All processes start execution at time** `0`, and are entered into the system in the order of the command line arguments (*with the third argument being the first process*).

```
java A3 F Q data1 data2 ... dataN
```

For example:

```
java A3 30 3 process1.txt process2.txt process3.txt
```

…where `30` is the number of frames (`F`) and `3` is the quantum size (`Q`) for Round Robin and related algorithms. Values `F` and `Q` are assumed to be integer values.

This is the only valid means of execution for this assignment.

The following arguments are text file names containing page references for each process, as defined in the next section. These are relative filenames and **SHOULD NOT BE HARDCODED** in anyway (*be it path, filename, extension, etc*). You may assume the number of supplied processes is a *minimum of two*, with no upper bound.

To be clear, the filenames above can be *any* filename with any extension (*or no extension*) – **do not hardcode filenames or extensions**! Please note that submissions that diverge from this will be **marked with heavy penalty**.

3.4.2. Input Notes

You may assume all arguments and data is 'sunny day' – we will not be testing with invalid values (*say a negative* `F` *value*) or invalid data files (*thus they will all follow the provided format, with white space to be ignored*) or invalid file paths – therefore you are not expected to do any validation.

4. Submission Guidelines

4.1. Deliverable

The following must be observed in your deliverable:

1. Your submission will contain your **program source code** with documentation and the report (*below*) in the root of the submission. These files will be **zipped and submitted in an archive** named **`c9876543.zip`** (*where* **`c9876543`** *is your student number*) – do not submit a `.rar`, or a `.7z`, or etc.

   If you vary from this format, **there will be a deduction** for non-standard submission.

2. Your **main class** will be named **`A3.java`**; your program will thus compile with the command **`javac A3.java`**, and your program will be executed by running **`java A3 30 3 process1.txt process2.txt ... processn.txt`**….where the files can be *any* filename – **do not hardcode filenames or extensions**!

   Note: If your program can not be compiled and executed in the expected manner (*above*) then please add a `readme.txt` (*containing any special instructions required to compile and run the source code*) in the root of the submission. Please note that such non-standard submissions will be **marked with penalty**.

3. Your **report(s)** – you will submit as **PDF document** called **`Report.pdf (and GenAIReport.pdf)`**, and place it in the root of the submission archive.

Variation from these deliverable standards will attract marking deductions.

4.2. Mark Distribution

Mark distribution can be found in the assignment draft marking breakdown documents: `26s2_Assign3_Feedback.pdf`.

Note: these are draft marking breakdowns and maybe updated.

4.3. Submission Notes

While a formality, please be aware of the following:

1. Assignments submitted after the deadline (**11:59 pm 8th November 2026)** will have the final result reduced by 10% of the maximum possible marks per day.

   A '*day*' begins at 12:00 Midnight, and once this time is passed, the penalty for the whole day is applied.

   This is UoN Policy.

2. If your assignment is not prepared and submitted following provide instructions then you will lose marks, in spite of parts of the task being correctly calculated.

3. If you have been granted an extension, please include a copy of the extension approval in as a **PDF document**, called `Extension.pdf`, and place this in the root of your submission, along with your other documents and code.